DT081A — Laboratory 3 Report

Student: Jakob Obid

2026-11-06

1 Introduction

This lab studies how pixel values, colour channels, filters and spatial patterns affect digital images.

2 Experimental Setup

I used Python, NumPy, OpenCV and Matplotlib. The main test image was `ManWoman.png`, with 631×475 pixels. For the first experiment, I converted it to grayscale and applied gamma correction. I also compared the Y, R, G and B channels and reduced the chroma resolution. I also tested 4-bit colour quantization and 3-bit grayscale quantization. I compared Gaussian filtering methods and used Sobel filters to detect image edges. I compared brightness and contrast adjustment using vectors and nested loops. I also compared direct pixel calculations with a lookup table. I also created elementary images using outer products and used three of them to approximate a triangle.

3 Results and Analysis

3.1 Human Perception of Color, Spatial Detail, and Gamma Correction

Q1.1. Implement gamma correction and calculate the outputs for intensities 64, 128 and 192 using gamma = 2.0.

Answer: I used gamma correction with gamma = 2.0. The outputs were 16, 64 and 144 for the three input values. The values were converted to integers for the final result.

Input	Output
64	16
128	64
192	144

Q1.2. Compare the image histograms before and after applying gamma = 2.2. Explain the relationship to nonlinear light perception.

Answer: I applied gamma = 2.2 to the grayscale image. The mean intensity fell from 95.28 to 46.25, and the histogram moved toward darker values (Figure 1). This shows how a nonlinear intensity change affects brightness, which relates to how the eye responds nonlinearly to light.

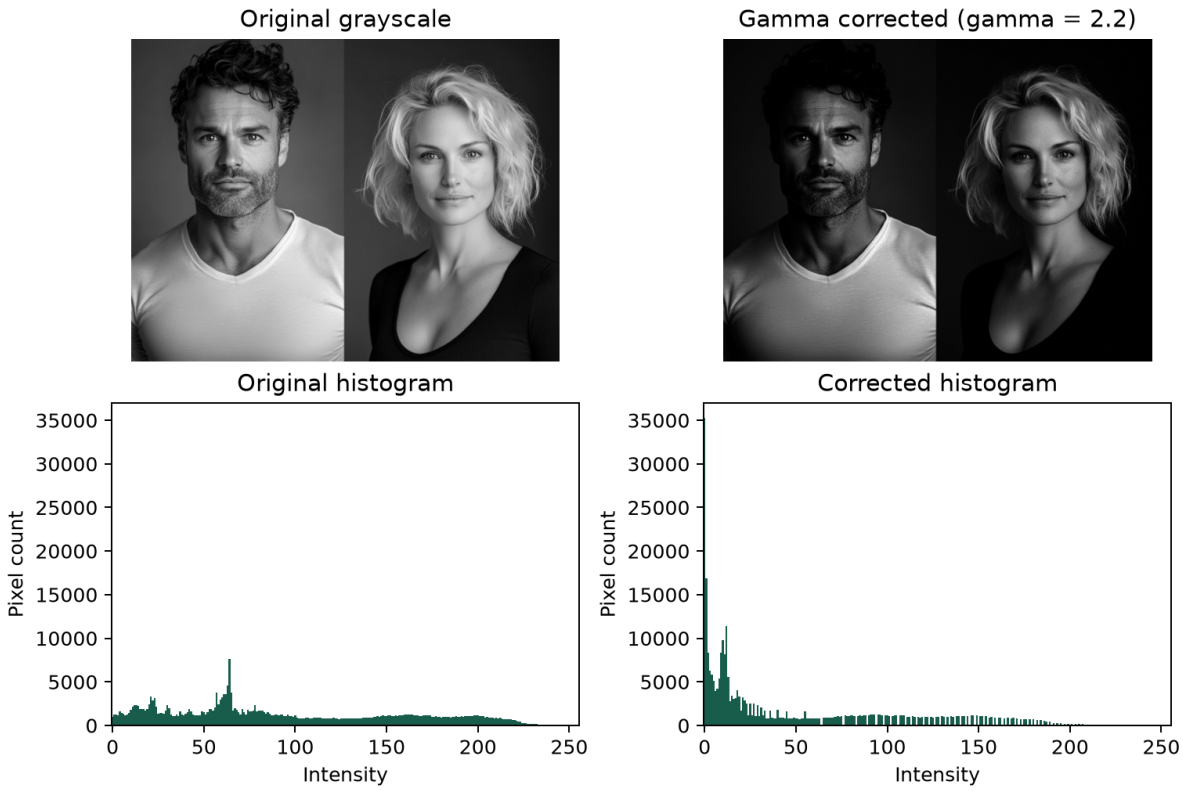


Figure 1: Original grayscale image and gamma-corrected image (gamma = 2.2), with their intensity histograms.

3.2 Colorspaces (RGB and YCbCr)

Q2.1. Compare the histogram spread of the Y channel versus R, G, and B channels. Why is the Y channel typically used for edge detection?

Answer: The R channel had the widest histogram, with a standard deviation of 70.20. The Y channel had a standard deviation of 63.89, between R and the G/B channels. Y is useful for edge detection because it represents brightness in one channel, although it may miss boundaries that only differ in colour (Figure 2).

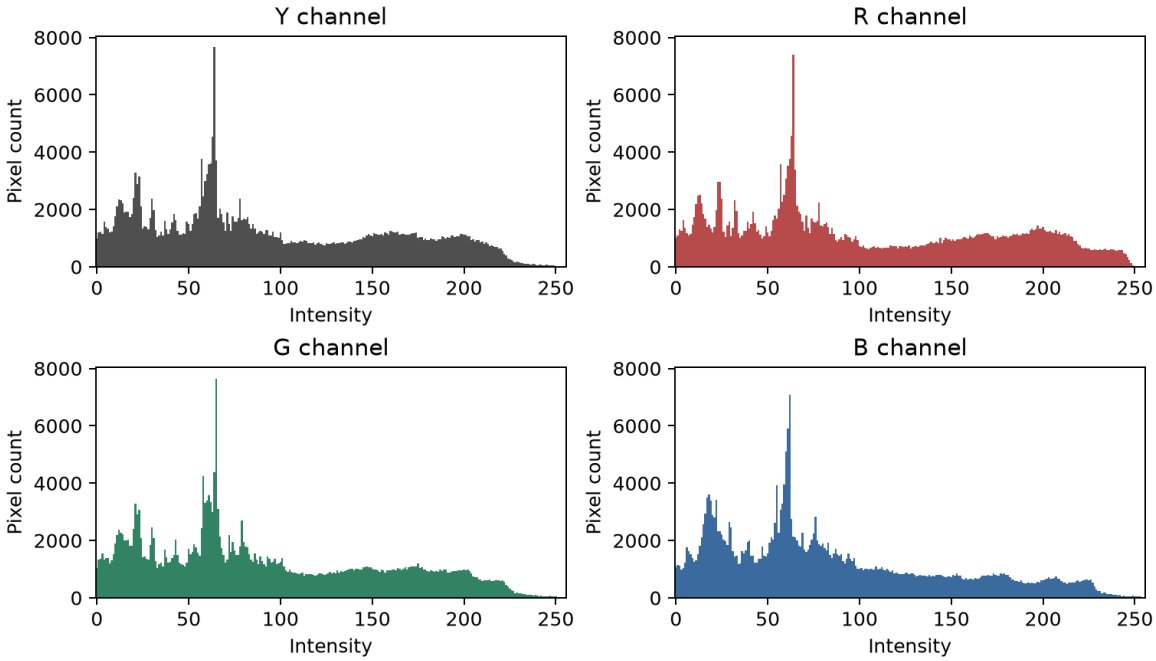


Figure 2: Intensity histograms of the Y, R, G and B channels.

Q2.2. If Cb and Cr channels are subsampled by a factor of 2, calculate the memory savings. At what point does this subsampling become visually noticeable if the subsampling factor is increased?

Answer: Reducing both chroma dimensions by a factor of 2 saved about 50% of the uncompressed channel storage. At factor 2, the image looked almost the same. At factors 4 and 8, colour edges became slightly softer when zoomed in (Figure 3).

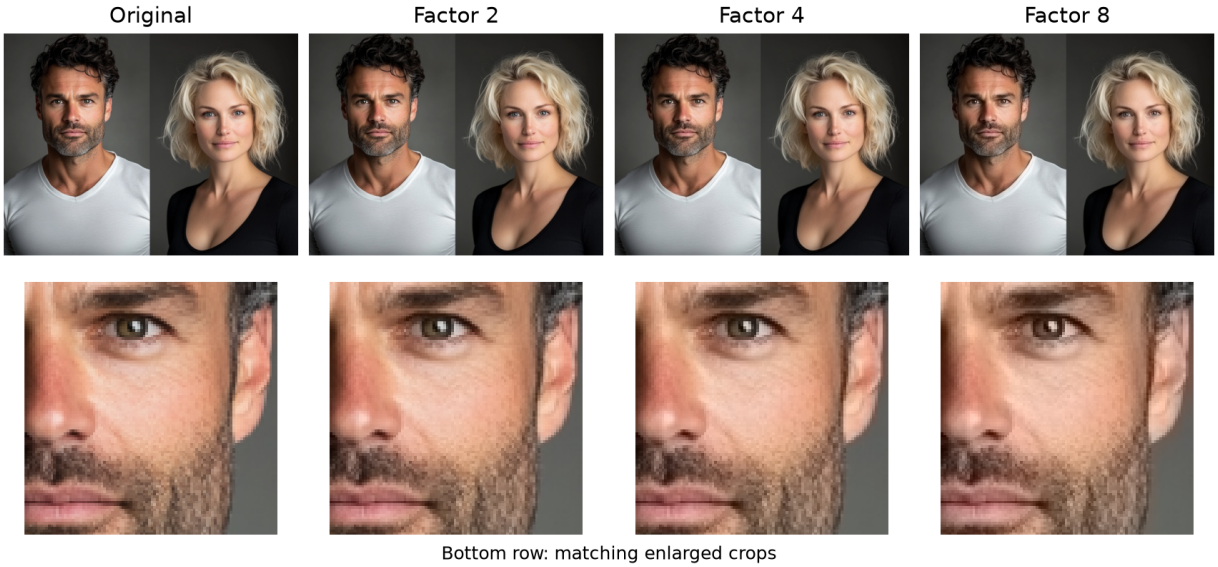


Figure 3: Chroma subsampling at factors 2, 4 and 8 compared with the original image.

3.3 Image Quantization and Sampling

Q3.1. Quantize the image to 4 bits and calculate the theoretical compression ratio achieved.

Answer: I reduced each colour channel from 8 bits to 4 bits, giving 16 possible levels per channel. The theoretical compression ratio was 2:1, which means 50% less storage if the values are packed. The colours became less smooth (Figure 4).

Q3.2. Plot the intensity distribution before and after 3-bit quantization. Explain the histogram's structure.

Answer: I reduced the grayscale image to eight intensity levels. The new histogram had eight groups because every 32 input values were combined into one level. This removed some of the original intensity detail (Figure 4).

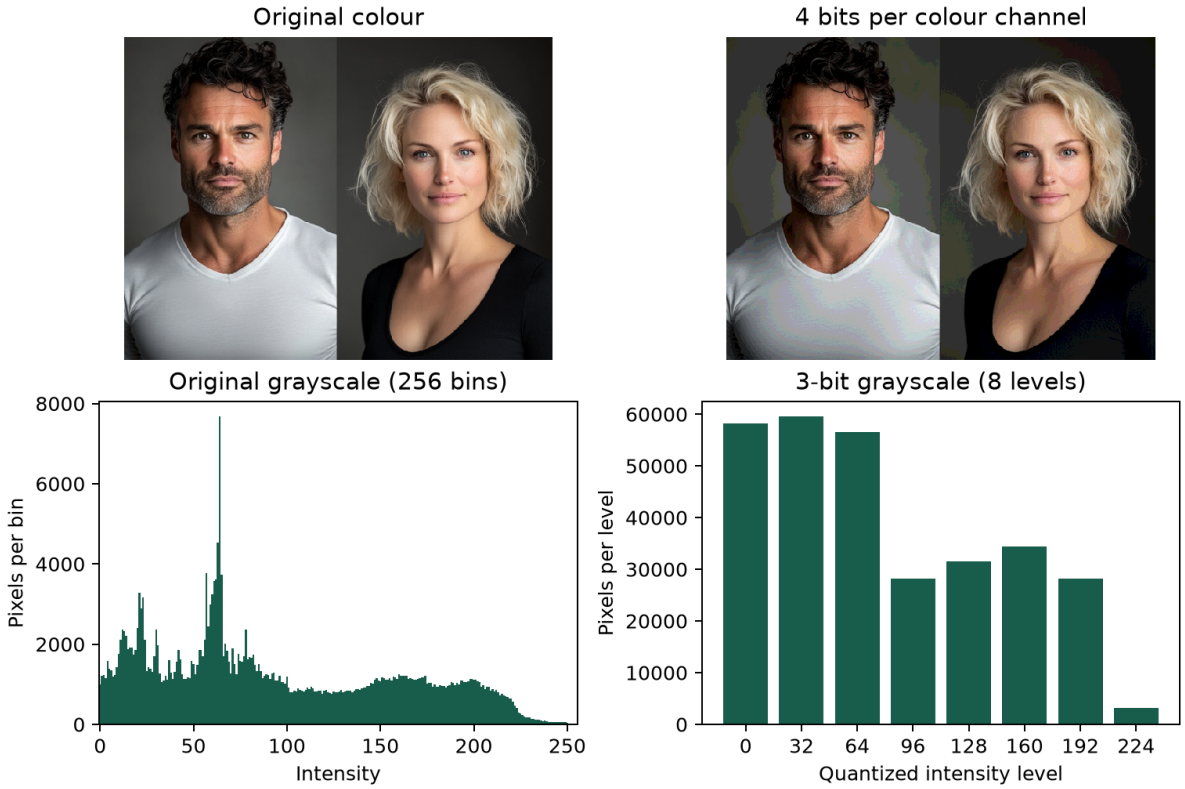


Figure 4: Original and quantized images, with grayscale intensity histograms before and after 3-bit quantization.

3.4 Point Spread Functions and Convolution

Q4.1. Compare the execution time of separable versus non-separable Gaussian filtering. When does separability provide significant advantage?

Answer: I compared normal 2D Gaussian filtering with separable filtering using three kernel sizes. The measured times are shown in Table 2. Both methods gave the same result within numerical precision. Separable filtering uses fewer kernel calculations for larger kernels, but the actual speed also depends on OpenCV.

Table 2: Median Gaussian filtering times.

Kernel	Ordinary 2D (ms)	Separable (ms)
5×5	5.149	2.225
11×11	5.148	2.922
21×21	8.635	6.218

Separable filtering was faster for all three tested kernel sizes.

Q4.2. Create a function that enhances an image's horizontal edges while suppressing vertical ones. Explain its structure.

Answer: I used the Sobel filter with `dx=0` and `dy=1` to detect horizontal edges. It finds brightness changes from top to bottom, so horizontal boundaries appear stronger while vertical ones are weaker (Figure 5).

Q4.3. Show the result of applying the magnitude gradient kernel to the image.

Answer: I combined the horizontal and vertical Sobel results to calculate the gradient magnitude. This highlighted edges in both directions (Figure 5).

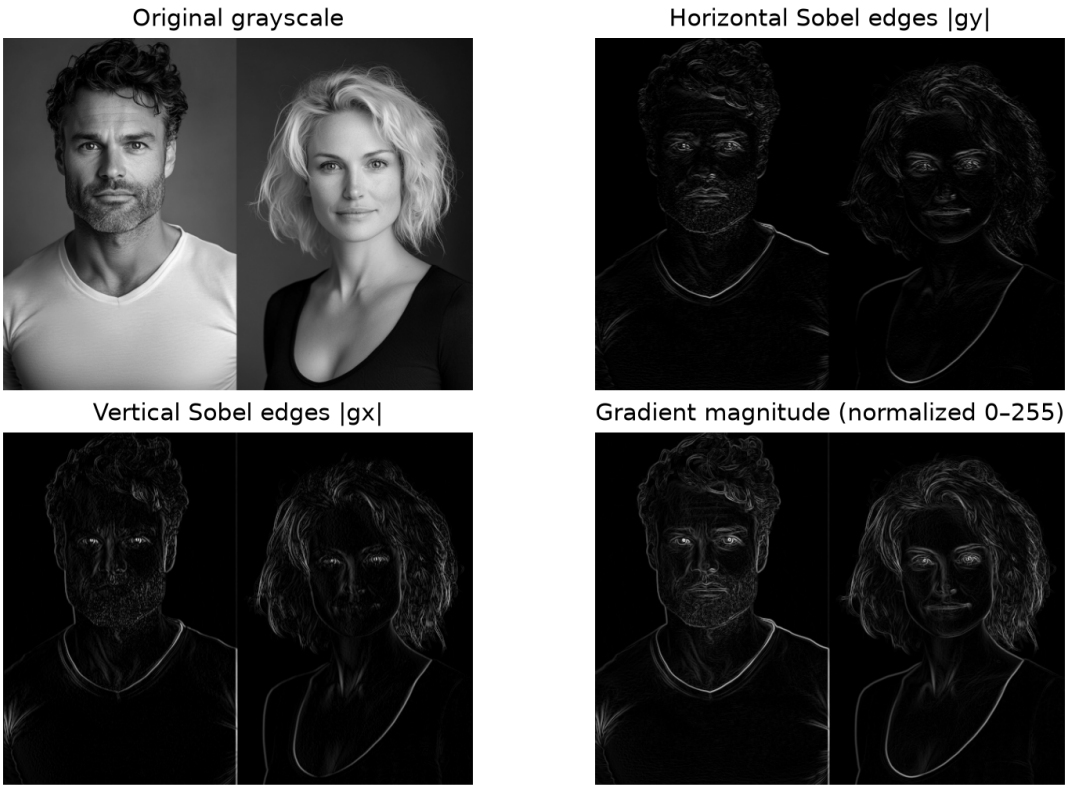


Figure 5: Original grayscale image, horizontal and vertical Sobel edges, and the combined gradient magnitude.

3.5 Images as Stacked Vectors and Matrix Operations

Q5.1. Implement image brightness and contrast adjustment using vector operations. Compare the computational efficiency with element-wise operations where nested loops are used, by measuring the execution time.

Answer: I adjusted the image using a flattened vector and nested Python loops. Both methods produced the same result. The vector method took 2.72 ms, while the loops took 199.59 ms. The

loop-to-vector time ratio was 73.45.

Method	Time (ms)
Vectorized NumPy	2.72
Nested loops	199.59

Q5.2. Given a 512×512 image, calculate the memory requirements for storing the explicit transformation matrix H for a 3×3 convolution operation.

Answer: A 512×512 image has 262144 pixels, so the full matrix H would contain 68719476736 elements. It would need 256 GiB using float32 or 512 GiB using float64. This is too much memory for a simple 3×3 convolution, so applying the small kernel directly is more practical.

3.6 Pixel-Based Processing

Q6.1. For your implementation, which is more efficient: direct computation or lookup tables? Quantify the difference.

Answer: I applied the same logarithmic transformation using direct NumPy calculations and a lookup table. Both methods produced the same image. Direct computation took 9.83 ms, while the lookup table took 1.32 ms. The speed ratio was 7.45.

Method	Average time (ms)
Direct NumPy	9.83
Lookup table	1.32

The time comparison excludes the one-time cost of building the lookup table.

Q6.2. Design a logarithmic transformation that enhances contrast in the middle intensity range while compressing shadows and highlights. Plot the transfer function.

Answer: I used a logarithmic transformation with strength 3. The curve rises faster near middle gray and more slowly near black and white. For example, input values 112 and 144 became 98 and 157. This increases contrast between middle brightness values (Figure 6).

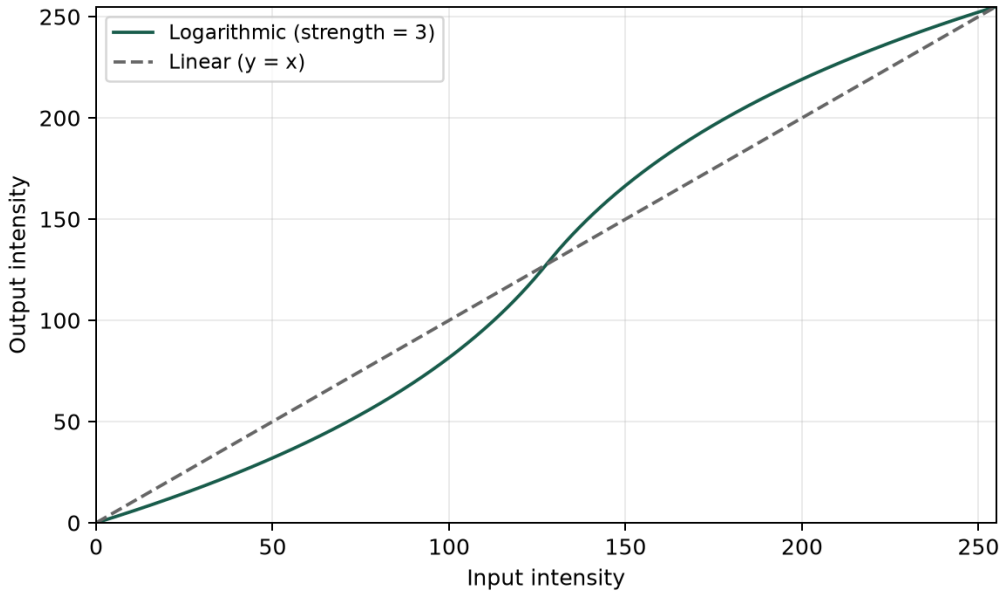


Figure 6: Logarithmic intensity transformation with increased middle-range contrast, compared with a linear transformation.

3.7 Elementary Images and Linear Superposition

Q7.1. Create four different elementary images of size 16×16 using combinations of ramp, step, and sine basis vectors. How do they capture different image features?

Answer: I created four 16×16 images using ramp, step and sine vectors. Step $\times$ Step made a sharp block, while Step $\times$ Ramp and Ramp $\times$ Step combined sharp boundaries with gradual changes. Sine $\times$ Sine created a repeating pattern (Figure 7).

Q7.2. Select a simple geometric shape in an image of size 16×16 and approximate it using only three elementary images by finding the best respective weights.

Answer: I created a small triangle and approximated it using three elementary images. The best weights were 0.5625, -0.4327 and 0.4327, with an MSE of 0.0521. The approximation showed some of the triangular shape, but the diagonal edge was not exact (Figure 7).

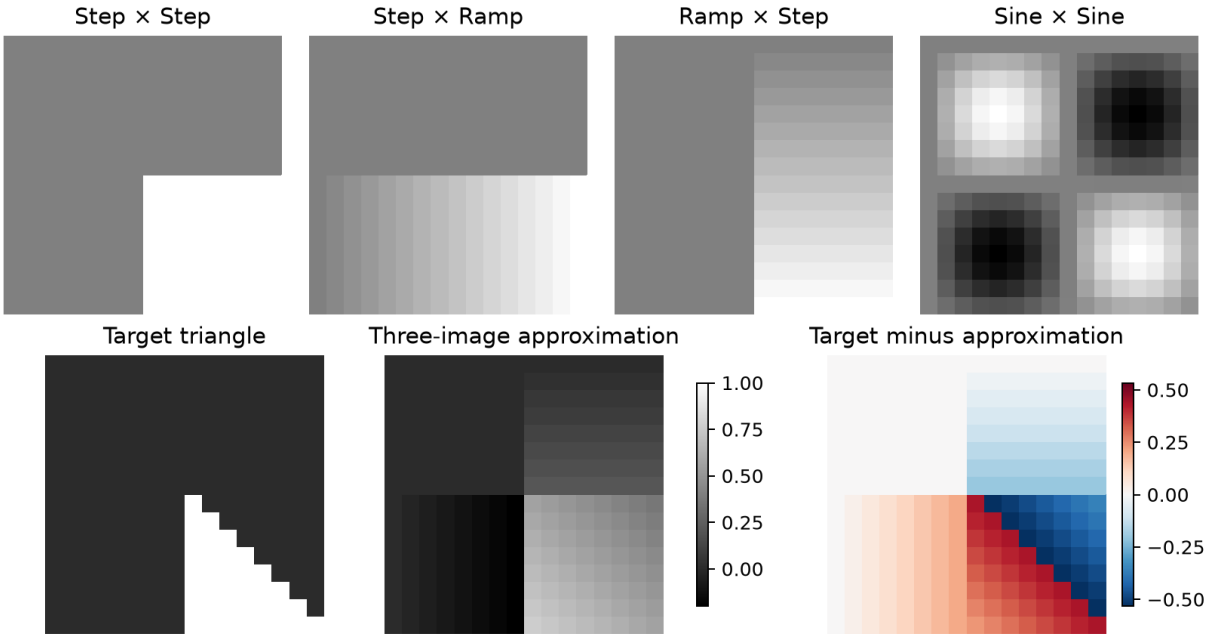


Figure 7: Four elementary 16×16 images and a triangle approximated using three weighted elementary images.

4 Discussion

Gamma correction made the image darker because gamma was greater than 1. Chroma subsampling saved storage while keeping most visible image detail. Reducing the number of intensity levels saved theoretical storage space but removed some image detail. Gaussian filtering showed how the same blur can be calculated in different ways. Sobel filtering highlighted edges based on brightness changes. Both brightness and contrast methods produced the same image. Their execution times showed the cost of processing pixels with Python loops. The lookup table stores the result for each possible input intensity, so the same calculation does not need to be repeated for every pixel. The triangle approximation showed that a few elementary images can reproduce some features, but not every detail.

5 Conclusion

The first experiment showed how changing gamma affects image brightness and its histogram. The colour-space experiment showed how brightness and colour information can be processed separately. Quantization showed how lower bit depth reduces the number of colours and intensity levels. The

convolution experiments showed how filters can blur images and detect edges. The matrix experiment showed why vectorized operations and small convolution kernels are useful. The pixel-processing experiment showed how intensity transformations and lookup tables work. The elementary-image experiment showed how images can be built from weighted patterns.